Отчёт по лабораторной работе №2

Фильтрация изображений на GPU: CUDA, libpng,
накладные расходы на организацию вычислений

Выполнил: Кардаш Руслан, группа 25224

1 Задание

Реализовать программу для наложения фильтров на изображения. Возможные фильтры: размытие, выделение границ, избавление от шума. Измерить время.

Для работы с графическими файлами рекомендуется использовать `libpng` (`man libpng`). Примеры использования библиотеки в `/usr/share/doc/libpng12-dev/examples/`.

Считать изображение из файла, преобразовать в массив, отправить на CUDA Device, провести на устройстве обработку фильтром, вернуть изображение в память хоста, преобразовать в картинку, сохранить файл. Оценить накладные расходы на организацию вычислений на GPU по сравнению с ЦПУ.

2 Фильтрация изображений на GPU

2.1 Постановка задачи

Дано изображение $I(x, y)$ размера $W \times H$. Требуется применить фильтр F, то есть для каждого пикселя вычислить новое значение на основе его окрестности. Основные операции:

- размытие (усреднение, гауссово размытие);
- выделение границ (оператор Собеля, Превитта);
- подавление шума (медианный фильтр).

2.2 Свёртка

Для линейных фильтров операция описывается свёрткой:

$$(I * K)(x, y) = \sum_{i=-r}^{r} \sum_{j=-r}^{r} K(i, j)\, I(x + i,\, y + j),$$

где K — ядро свёртки. Для гауссова размытия 3×3:

$$K_{\text{blur}} = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}.$$

Оператор Собеля использует два ядра:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}.$$

Модуль градиента: $|\nabla I| = \sqrt{(G_x * I)^2 + (G_y * I)^2}$.

2.3 Медианный фильтр

Нелинейный фильтр: для каждого пикселя берётся окрестность 3×3, значения сортируются, и на выход подаётся медиана (5-й элемент). Хорошо удаляет импульсный шум.

2.4 Реализация на CUDA

Изображение представляется как массив `uchar4` (RGBA). Каждый поток обрабатывает один пиксель. Для эффективного доступа к окрестности используется разделяемая память (shared memory). Блок 16×16, тайл 18×18. Загрузка тайла с зажимом координат (clamp) для обработки границ.

2.5 libpng

Библиотека libpng предоставляет упрощённый API:

```
png_image image;
image.version = PNG_IMAGE_VERSION;
png_image_begin_read_from_file(&image, filename);
image.format = PNG_FORMAT_RGBA;
png_image_finish_read(&image, NULL, buffer, 0, NULL);
```

Аналогично для записи.

2.6 Оценка накладных расходов

Время GPU-обработки складывается из:

$$T_{\text{GPU}} = T_{\text{malloc}} + T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}}.$$

Накладные расходы на организацию вычислений:

$$T_{\text{overhead}} = T_{\text{H2D}} + T_{\text{D2H}} + T_{\text{malloc}}.$$

Ускорение:

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}.$$

По закону Амдала, если доля последовательных операций f, то максимальное ускорение ограничено $1/f$. Для больших изображений T_{kernel} доминирует, и ускорение приближается к отношению вычислительных мощностей. Для малых изображений накладные расходы на копирование могут превысить выигрыш.

2.7 Измерения

Для каждого фильтра измеряются:

- T_{CPU} — время CPU-реализации;
- T_{H2D} — копирование хост $\rightarrow$ устройство;
- T_{kernel} — время ядра;
- T_{D2H} — копирование устройство $\rightarrow$ хост;
- $T_{\text{GPU}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}}$.

Выводится ускорение и доля накладных расходов.

3 Численные результаты

3.1 Прогон на Tesla T4: изображение 512×512

Эксперимент проводился для изображения 512×512 (grayscale) на GPU NVIDIA Tesla T4, CUDA 13.0. Время усреднялось по 10 запускам после прогрева.

Таблица 1. Время CPU- и GPU-реализаций, накладные расходы и ускорение (Tesla T4, 512×512).

Фильтр	T_{CPU}, мс	T_{kernel}, мс	T_{GPU}, мс	T_{overhead}, мс	S_{kernel}, раз	S_{total}, раз	diff > 1
Blur	4.48814	0.0156352	0.191030	0.175395	287.054	23.4944	0
Sobel	4.47687	0.0130272	0.182372	0.169345	343.656	24.5480	0
Median	7.35370	0.0759456	0.251380	0.175434	96.8285	29.2533	0

Выводы:

- GPU-ядро обеспечивает ускорение фильтрации в 97–344 раза; с учётом копирований H2D/D2H итоговое ускорение составляет 23–29 раз.
- Для изображения 512×512 накладные расходы на передачу данных и организацию вычислений (≈ 0.17–0.18 мс) сравнимы с полным GPU-временем и доминируют над временем ядра (0.013–0.076 мс).
- Медианный фильтр имеет самое медленное ядро из-за сортировки окрестности, но даёт наибольшее итоговое ускорение за счёт более дорогой CPU-реализации.
- Для малых изображений накладные расходы ограничивают выигрыш; с ростом размера изображения вклад T_{kernel} становится определяющим.

3.2 Прогон на Quadro RTX 4000: изображение 512×512

Эксперимент проводился на вычислительном узле кластера НОЦ «Газпромнефть-НГУ»: NVIDIA Quadro RTX 4000, 16 ядер CPU, 80 ГБ ОЗУ, Oracle Linux 8, CUDA Toolkit 12.3. Изображения — синтетические, 8-битные grayscale, генерировались самой программой и подавались на вход одинаково для CPU- и GPU-реализаций. Время усреднялось по 10 запускам после прогрева; для ядра использовались события CUDA, для CPU — `std::chrono`. Корректность подтверждается столбцом *diff* — числом пикселей, отличающихся от CPU-результата более чем на 1.

Таблица 2. Результаты для изображения 512×512 (Quadro RTX 4000).

Фильтр	T_{CPU}, мс	T_{kernel}, мс	T_{GPU}, мс	T_{overhead}, мс	S_{kernel}, раз	S_{total}, раз	diff > 1
Blur	3.182	0.459	4.793	4.334	6.9	0.7	0
Sobel	6.538	0.454	4.691	4.237	14.4	1.4	0
Median	9.013	0.492	4.372	3.880	18.3	2.1	0

При малом объёме данных накладные расходы $T_{\text{overhead}} \approx 3.9$–$4.3$ мс сопоставимы с временем CPU-реализации (3–9 мс), поэтому полное ускорение S_{total} не превышает 1.4–2.1, а для blur GPU-версия даже проигрывает CPU ($S_{\text{total}} = 0.7$). Несмотря на то что само ядро работает в 7–18 раз быстрее CPU, пересылки данных H2D/D2H и синхронизации полностью съедают этот выигрыш.

3.3 Изображение 4096×4096

Таблица 3. Результаты для изображения 4096×4096 (Quadro RTX 4000).

Фильтр	T_{CPU}, мс	T_{kernel}, мс	T_{GPU}, мс	T_{overhead}, мс	S_{kernel}, раз	S_{total}, раз	diff > 1
Blur	194.323	0.732	10.122	9.390	265.3	19.2	0
Sobel	434.823	0.754	9.501	8.747	576.8	45.8	0
Median	573.675	8.225	16.799	8.574	69.8	34.1	0

При увеличении площади изображения в 64 раза время CPU-реализации выросло пропорционально, тогда как накладные расходы GPU T_{overhead} выросли лишь в 2.2–2.4 раза (с ≈ 4 до ≈ 9 мс): для 4096×4096 (16 МБ данных) больший вклад дают собственно пересылки, а не латентность драйвера. В результате T_{overhead} теперь на порядок меньше T_{CPU}, и полное ускорение достигает 19–46 раз. Наибольший выигрыш ($S_{\text{total}} = 45.8$) показывает sobel — у него высокая арифметическая интенсивность на пиксель и хорошо распараллеливаемая структура. У медианного фильтра ускорение ниже ($S_{\text{total}} = 34.1$), так как на каждый пиксель приходится сортировка окна из 9 элементов, что заметно дороже в расчёте на один поток.

3.4 Сравнение размеров изображения

Таблица 4. Сводная таблица: влияние размера изображения (Quadro RTX 4000).

Фильтр	Размер	T_{CPU}, мс	T_{GPU}, мс	T_{overhead}, мс	S_{total}, раз
Blur	512^2	3.182	4.793	4.334	0.7
Blur	4096^2	194.323	10.122	9.390	19.2
Sobel	512^2	6.538	4.691	4.237	1.4
Sobel	4096^2	434.823	9.501	8.747	45.8
Median	512^2	9.013	4.372	3.880	2.1
Median	4096^2	573.675	16.799	8.574	34.1

Два прогона наглядно демонстрируют ответ на вопрос о накладных расходах. При малом объёме данных (512×512, ≈ 0.25 МБ) доминирует фиксированная стоимость организации вычислений на GPU — подготовка контекста, пересылки H2D/D2H, синхронизации — и она сопоставима с временем работы CPU; полное ускорение невелико, а для blur даже отрицательно. При росте изображения до 4096×4096 (≈ 16 МБ) время полезной работы GPU растёт как $O(W \cdot H)$, а накладные расходы — почти константа, поэтому GPU выходит вперёд с запасом в десятки раз. Точка окупаемости для этой конфигурации лежит где-то в диапазоне 10^5–10^6 пикселей.

4 Общий вывод

Тем самым цель работы — «оценить накладные расходы на организацию вычислений на GPU по сравнению с ЦПУ» — достигнута: они измерены напрямую ($T_{\text{overhead}} = T_{\text{GPU}} - T_{\text{kernel}}$) и проиллюстрированы на двух характерных масштабах задачи.